

MOVELINK

System-Dokumentation & Traceability-Report

Anforderungs- & Abdeckungsanalyse der MoveLink App

Status: Freigegeben

Sprache: Deutsch

Autoren: Erlind & AI Assistant Antigravity

Dokumentenversion: v1.1.0 (Scraped Live)

Veröffentlichungsdatum: 20. Juni 2026

Gliederung (Inhaltsverzeichnis)

Dieses Dokument gliedert sich in die folgenden Abschnitte:

1. Anwendungsfälle (Use Cases)	3
2. Systemanforderungen (Requirements)	5
Funktionale Anforderungen //Was für Instanzen existieren denn eigentlich.	5
Nicht-funktionale Anforderungen	5
Randbedingungen	5
3. Architekturentscheidungen (ADR / Abwägungen)	7
Abwägungen	7
4. System-Architektur (C4 Modell)	9
4.1 Embedded Sensor-Firmware (Xiao MCU)	9
4.2 Mobile App (React Native)	17
C4-Architektur-Ebene	17
Beschreibung	17
Requirements	18
Komponenten in diesem Container	18
4.3 Datenbank & Backend (PostgreSQL & Node.js)	20
C4-Architektur-Ebene	20
Beschreibung	20
Requirements	21
Komponenten in diesem Container	21
5. Rückverfolgbarkeits-Matrix (Traceability)	22

1. Anwendungsfälle (Use Cases)

In diesem Abschnitt werden die primären Anwendungsfälle (Use Cases) des MoveLink-Systems beschrieben. Diese Use Cases definieren die Benutzerinteraktionen und Systemantworten für das Verbinden von Geräten, das Echtzeit-Training und das Einsehen historischer Daten.

Hier werden die primären Interaktionen zwischen dem Trainierenden und dem MoveLink-System beschrieben.

UC-1 – Trainingsgerät verbinden

- **Akteur:** Trainierender
- **Vorbedingung:** Trainingsgerät ist eingeschaltet und befindet sich in Reichweite.
- **Beschreibung:** Als Trainierender möchte ich mein Trainingsgerät mit der App verbinden, um Trainingsdaten erfassen zu können.
- **Ablauf (Szenario):**
 1. **Eingabe:** Der Trainierende öffnet die App.
- **Ausgabe*:** Die App zeigt eine Möglichkeit/einen Reiter für Hardwaregeräte an.
- 2. **Eingabe:** Der Trainierende klicke auf den Reiter "Hardwaregeräte".
- **Ausgabe*:** Die App zeigt eine Liste der verfügbaren Hardwaregeräte sowie den aktuellen Verbindungsstatus an.
- 3. **Eingabe:** Der Trainierende wählt ein Hardwaregerät aus der Liste aus und klickt auf "Verbinden".
- **Ausgabe*:** Die App zeigt die Detailansicht des ausgewählten Geräts und bestätigt den erfolgreichen Verbindungsaufbau.

UC-2 – Echtzeit-Training überwachen

- **Akteur:** Trainierender
- **Vorbedingung:** Das Trainingsgerät ist erfolgreich mit der App verbunden.
- **Beschreibung:** Ich als Trainierender möchte mein Training in Echtzeit überwachen können, um direkt Feedback zu meiner Ausführung zu erhalten.
- **Ablauf (Szenario):**
 1. **Eingabe:** Der Trainierende öffnet die App.
- **Ausgabe*:** Die App zeigt die Option zum Starten eines Trainings an.
- 2. **Eingabe:** Der Trainierende klickt auf "Training starten".
- **Ausgabe*:** Die App zeigt die Detailansicht des ausgewählten Trainings sowie die Start-Schaltfläche.
- 3. **Eingabe:** Der Trainierende drückt den "Start"-Button.
- **Ausgabe:** Die App demonstriert grafisch die auszuführende Übungsbewegung. (Bezug: **FA8**)*

4. **Eingabe:** Der Trainierende führt die Bewegung aus.

- **Ausgabe:** Die App visualisiert die Bewegung in Echtzeit, vergleicht sie mit der Zielvorgabe und gibt positives Feedback bei korrekter Ausführung. (Bezug: FA5, FA6, FA9)*

UC-3 – Trainingsdaten einsehen

- **Akteur:** Trainierender
- **Vorbedingung:** Mindestens eine aufgezeichnete Trainingseinheit ist in der Datenbank vorhanden.
- **Beschreibung:** Ich als Trainierender möchte vergangene Trainingseinheiten einsehen können, um meine Fortschritte zu verfolgen.
- **Ablauf (Szenario):**
 1. **Eingabe:** Der Trainierende öffnet die App.
 - **Ausgabe*:** Die App bietet eine Option zum Einsehen des Trainingsverlaufs.
 2. **Eingabe:** Der Trainierende navigiert zum Reiter "Trainingseinheiten".
 - **Ausgabe*:** Die App listet alle vergangenen Trainingseinheiten chronologisch auf.
 3. **Eingabe:** Der Trainierende wählt eine Trainingseinheit aus der Liste aus.
 - **Ausgabe*:** Die App bereitet die historischen Bewegungsdaten grafisch und statistisch auf.

2. Systemanforderungen (Requirements)

Dieser Abschnitt enthält die funktionalen Anforderungen (FA), nicht-funktionalen Anforderungen (NF) und Randbedingungen (R) des MoveLink-Systems.

Dieses Dokument definiert die funktionalen und nicht-funktionalen Anforderungen sowie die Randbedingungen des MoveLink-Systems.

Funktionale Anforderungen //Was für Instanzen existieren denn eigentlich.

FA1 – Es wird eine Applikation angeboten, welche als Input für die Steuerung für den Benutzer dient. (UC-1, UC-2, UC-3)

FA2 – Zu dem System wird ein Trainingsgerät benötigt, welche als Input für der Bewegungen des Benutzers dient. Dieses Gerät gibt die Sensorwerte als auch die Art der Bewegung zurück. (UC-1, UC-2)

FA3 – Es wird eine Datenbank benötigt, welche die Daten des Benutzers speichert. (UC-1, UC-3)

Nicht-funktionale Anforderungen

NF1 – Latenz

Die End-to-End-Latenz von der physischen Sensorbewegung bis zur visuellen Darstellung in der App muss ≤ 100 ms sein.

NF2 – Zuverlässigkeit und Reconnect

Bei Verbindungsabbrüchen muss die App den Nutzer umgehend benachrichtigen und automatische Wiederverbindungsversuche (Reconnect) starten.

NF3 – Benutzbarkeit (Usability)

Das Bluetooth-Pairing mit dem Sensor darf maximal zwei manuelle Interaktionen erfordern.

Randbedingungen

R1 – Plattform-Kompatibilität

Die mobile Applikation muss nativ oder als hybride App auf Android-Geräten lauffähig sein.

3. Architekturentscheidungen (ADR / Abwägungen)

In diesem Abschnitt werden die zentralen Architekturentscheidungen (Architecture Decision Records) und Abwägungen bezüglich Hardware, App-Framework und Auswertungstechnologie dokumentiert.

Abwägungen

Hardware/Gerät:

vorgefertigtes Gerät (Fitbit)	Hybrid (XIAO seed NRF52840)	Eigene Lösung (PCB)
+ Klein und ausgereift	+ Kostengünstig, integrierte IMU & BLE	+ Maximale Kontrolle über Formfaktor und Sensoren
- Teuer	- Gehäuse muss selbst konstruiert/gedruckt werden	- Sehr hohe Entwicklungskosten und Time-to-Market
- Schlecht erweiterbar durch andere Sensoren	- Höherer Integrationsaufwand als fertiges Konsumentenprodukt	- Komplexes PCB-Design und Fertigungsrisiko

✓ Entscheidung: XIAO seed NRF52840

App:

Ionic (Flutter)	Hybrid (React Native)	Native IOS Swift/Android Java
+ Schnelles Prototyping und einfache Web-Technologien	+ Hohe Code-Wiederverwendbarkeit und schnelle UI-Iterationen	+ Optimale Bluetooth-Leistung und direkter API-Zugriff
- Performance-Einbußen bei 50Hz Echtzeit-Diagrammen	- BLE-Bibliotheken von Drittanbietern erfordern Wartung	- Hohe Entwicklungskosten durch zwei separate Codebases
- Komplexere native Bluetooth-Anbindung über Plugins	- Performance-Overhead durch JS-Bridge bei kontinuierlichem Datenstrom	- Längere Time-to-Market und aufwendige doppelte Pflege

✓ Entscheidung: Hybrid (React Native)

Bewegungsauswertung:

Die Bewertung

Auf dem Gerät	Komplett in der App	Hybrid
+ Per Edge Impulse einfach umzusetzende Inferenz	+ Genug Rechenleistung für komplexe Deep-Learning-Modelle	+ MCU filtert/komprimiert Daten; App übernimmt Inferenz

- Begrenzte Speicher- und Rechenkapazitäten der MCU	- Hohe BLE-Datenrate (50Hz Rohdatenstrom) erforderlich	- Höhere Systemkomplexität durch geteilte Logik
- Modell-Updates erfordern Firmware-Flashen	- Erhöhter Akkuverbrauch auf dem Mobilgerät	- Komplexeres Debugging bei Übertragungsverzögerungen

✓ **Entscheidung: Auf dem Gerät (Edge-Inferenz)**

4. System-Architektur (C4 Modell)

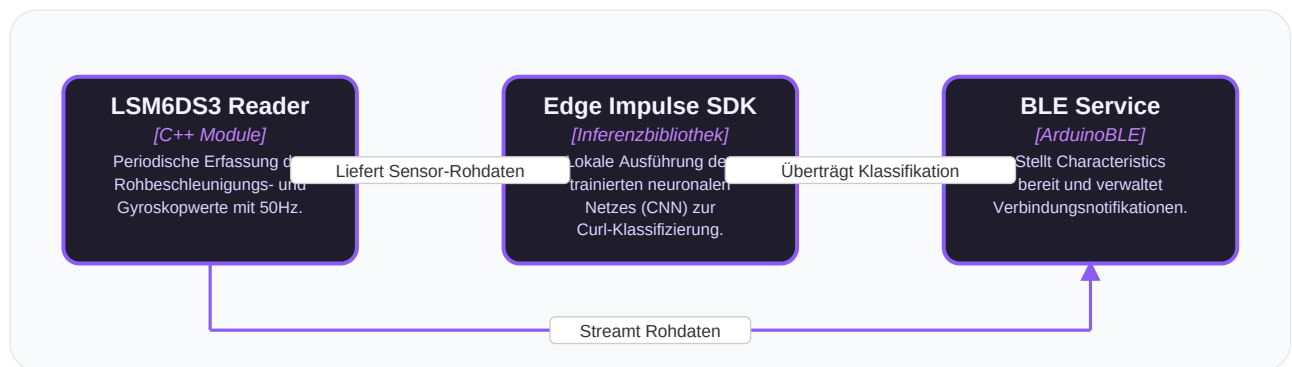
Die Software-Architektur von MoveLink ist nach dem C4-Modell strukturiert. Im Folgenden werden die einzelnen Container des Systems (Embedded Firmware, Mobile App und Datenbank & Backend) im Detail beschrieben. Für jeden dieser Unterpunkte werden die C4-Komponenten und Datenflüsse visuell und textuell dargestellt, gefolgt von den Anforderungen und der Architekturentscheidung (ADR).

4.1 Embedded Sensor-Firmware (Xiao MCU)

Die Sensor-Firmware läuft auf dem **XIAO nRF52840 Sense Controller**. Sie erfasst Beschleunigungs- und Rotationsdaten über den LSM6DS3-Sensor, wendet Filter an und überträgt Daten per BLE oder wertet sie per Edge-Inferenz direkt auf dem Chip aus.

Architektur-Abbildung (Komponenten)

Das folgende Komponenten-Diagramm zeigt die inneren Module der Sensor-Firmware sowie die Kopplung von Sensor-Loop, Edge Impulse Inferenz und dem BLE Service:



4.1.1 Container-Architektur & Beschreibung

Dieses Dokument beschreibt die Embedded Sensor-Firmware als eigenständige, deploybare Einheit im C4-Modell.

C4-Architektur-Ebene

- **C4-Ebene:** Container
- **Deployable:** Ja
- **Deployment-Artefakt:** Binär-Firmware (flashed via USB/Serial)
- **Technologie-Stack:** Arduino C/C++, LSM6DS3 IMU Library, Edge Impulse SDK, Bluetooth Low Energy

Beschreibung

Die Sensor-Firmware läuft auf dem XIAO nRF52840 Sense Controller. Sie erfasst Beschleunigungs- und Rotationsdaten über den integrierten LSM6DS3-Sensor mit einer festen Abtastrate (50Hz), wendet Signalfilterungen zur Rauschunterdrückung an und streamt die Datenpakete als binäres Array via BLE

Characteristics an die Mobile App. Alternativ führt sie Edge-Impulse-Inferenzmodelle direkt auf dem Mikrocontroller aus, um Trainingsübungen (z.B. Bizeps-Curls, Schulterdrücken und Seitheben) lokal zu klassifizieren und Fehler über die integrierten RGB-LEDs anzuzeigen.

Requirements

FA2.1 – Das Gerät sammelt die Dreh- und Beschleunigungsdaten des Trainierenden.

FA2.2 – Das Gerät erkennt, was für eine Bewegung ausgeführt worden ist.

FA2.3 – Das Gerät bewertet die Ausführung der Bewegung.

FA2.4 – Das Gerät gibt durch die LED den Verbindungsstatus aus.

FA2.5 – Das Gerät versendet die Daten an die App.

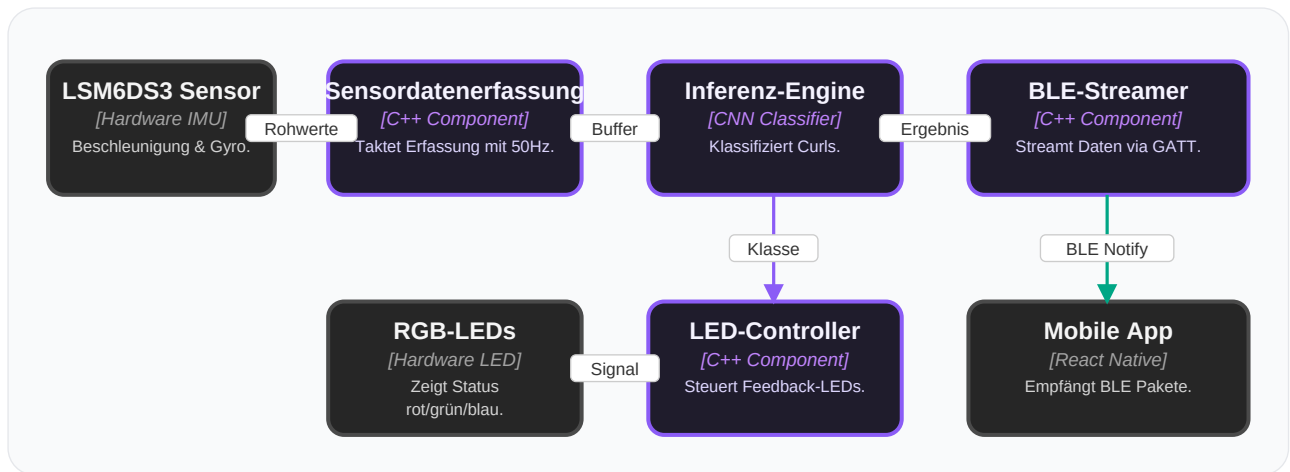
FA2.6 – Das Gerät ist in einem Gehäuse.

Komponenten in diesem Container

Die Sensor-Firmware besteht aus folgenden logischen Komponenten:

- **FA2.1** -> [Sensordatenerfassung (Loop)](file:///home/arch/repo/movelink/embedded/components/sensordatenerfassung/architecture.md)*: Liest kontinuierlich Beschleunigung (X, Y, Z) und Gyroskop (X, Y, Z).
- **FA2.2, FA2.3** -> [Inferenz-Engine (Edge Impulse)](file:///home/arch/repo/movelink/embedded/components/inferenz_engine/architecture.md)*: Klassifiziert Übungsausführungen lokal auf dem Chip.
- **FA2.4** -> [LED- & Display-Controller](file:///home/arch/repo/movelink/embedded/components/led_display_controller/architecture.md)*: Bietet direktes visuelles Feedback an den Nutzer bei Fehlern über RGB-LEDs.
- **FA2.5** -> [BLE-Streamer](file:///home/arch/repo/movelink/embedded/components/ble_streamer/architecture.md)*: Überträgt die erfassten Daten an den App-Container.
- **FA2.6** -> [Gehäuse](file:///home/arch/repo/movelink/embedded/components/gehaeuse/architecture.md)*: Bietet physischen Schutz, sodass das Tragen erleichtert wird.

Datenfluss



4.1.2 Sensordatenerfassung (Loop)

Diese Komponente liest kontinuierlich die Rohwerte des Beschleunigungssensors und Gyroskops aus.

C4-Architektur-Ebene

- **C4-Ebene:** Component
- **Deployable:** Nein (Läuft als Teil des Sensor Firmware Containers)

Beschreibung

Die Sensordatenerfassung liest in einer festen Schleife (Loop) die 6-Achsen-IMU-Werte (Beschleunigung und Drehung) des LSM6DS3-Sensors über den I2C-Bus ein.

Requirements

FA2.1 – Das Gerät sammelt die Dreh- und Beschleunigungsdaten des Trainierenden.

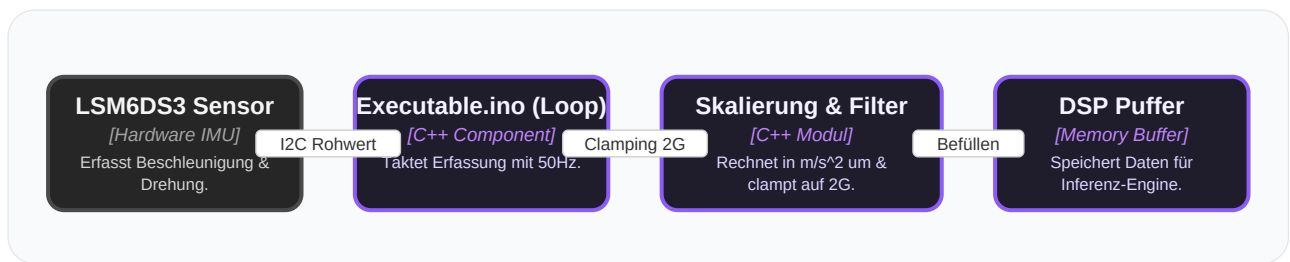
FA2.1.1 – Beschleunigung wird eingelesen

FA2.1.2 – Drehung wird eingelesen

Technische Details

- **Abtastrate:** 50 Hz (gesteuert durch präzises Timing in Microsekunden)
- **Signal-Clamping:** Beschleunigungswerte werden auf max. ± 2.0 G gedämpft/geclamt.
- **Konvertierung:** Die Werte werden in die SI-Einheit m/s^2 konvertiert ($1\text{G} = 9.80665\text{m/s}^2$).
- **Verwendete Hardware:** LSM6DS3 IMU auf dem XIAO nRF52840 Sense.

Datenfluss



4.1.3 Inferenz-Engine (Edge Impulse)

Diese Komponente klassifiziert Übungsausführungen in Echtzeit direkt auf dem Mikrocontroller (Edge Computing).

C4-Architektur-Ebene

- **C4-Ebene:** Component
- **Deployable:** Nein (Läuft als Teil des Sensor Firmware Containers)

Beschreibung

Die Inferenz-Engine führt ein CNN-Klassifikationsmodell aus, das über Edge Impulse trainiert und als Arduino-Bibliothek in die Firmware integriert wurde. Es analysiert die 6-Achsen-Bewegungsdaten auf spezifische Übungsqualitäten und Fehlerbilder.

Requirements

FA2.2 – Das Gerät erkennt, was für eine Bewegung ausgeführt worden ist.

FA2.3 – Das Gerät bewertet die Ausführung der Bewegung.

FA2.2.1 – Das Gerät erkennt einen Idle Modus

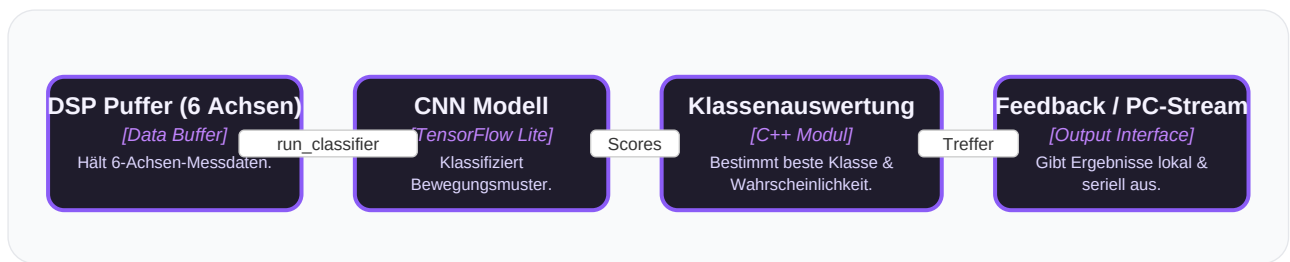
FA2.2.2 – Das Gerät erkennt einen Curl

FA2.2.3 – Das Gerät erkennt einen shoulder press

FA2.2.4 – Das Gerät erkennt einen Lateral raise

FA2.2.5 – Das Gerät erkennt eine tricep extension

Datenfluss



Technische Details

- **Modelltyp:** Convolutional Neural Network (CNN)
- **Erkannte Klassen:**
 - `idle` / `Idle`: Keine Übungsausführung / Ruhezustand.
 - `curl` / `curl_sauber` / `CURL`: Korrekt ausgeführter Bizeps-Curl.
 - `ShoulderPress` / `shoulderpress`: Schulterdrücken.
 - `lateralraise` / `LateralRaise` / `LateralRaises`: Seitheben.
 - `fehler_rotation`: Fehlerhafte Ausführung des Bizeps-Curls durch Rotation des Handgelenks.
 - `fehler_ellbogen`: Fehlerhafte Ausführung des Bizeps-Curls durch Bewegung des Ellbogens.
- **Anomalieerkennung:** Optionaler K-Means-Clustering-Block zur Erkennung unbekannter Bewegungen.

4.1.4 LED- & Display-Controller

Diese Komponente gibt dem Trainierenden direktes visuelles Feedback zur Qualität und dem Typ der Übungsausführung über die integrierte RGB-LED.

C4-Architektur-Ebene

- **C4-Ebene:** Component
- **Deployable:** Nein (Läuft als Teil des Sensor Firmware Containers)

Beschreibung

Der Controller steuert die integrierten Low-Active RGB-LEDs des XIAO-Boards (Pins LEDR, LEDB und LEDG) basierend auf den Klassifikationsergebnissen der Inferenz-Engine. Das ursprüngliche OLED-Display (U8x8) wurde aus Designgründen entfernt, um Gewicht und Energiebedarf einzusparen.

- *Feedback-Logik**
- **Warte auf Bluetooth-Verbindung (kein BLE verbunden):**
- **RGB-LED: Langsames Blinken (Blau)** (Pin LEDB blinkt an/aus)
- **Ruhemodus (`idle` oder Konfidenz < 60%):**

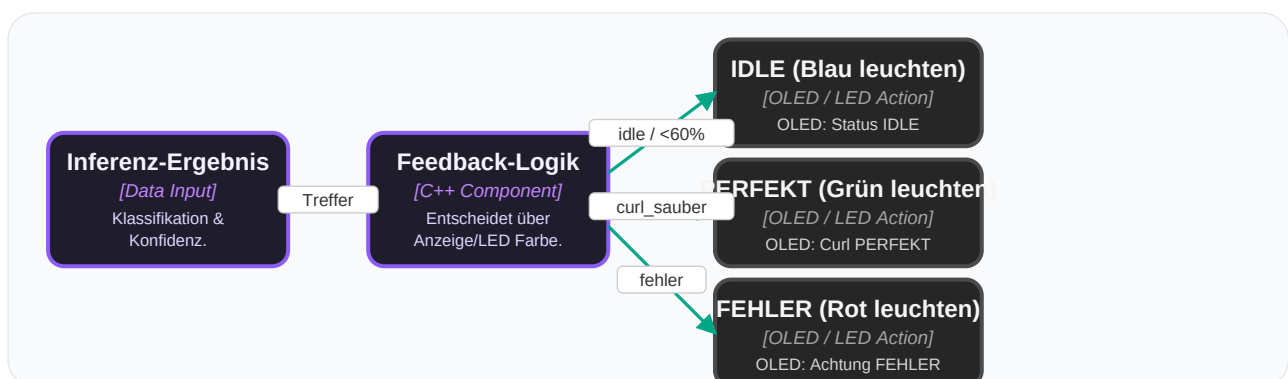
- RGB-LED: **Blau** (Pin LEDB auf LOW)
- **Schulterdrücken** (ShoulderPress):
- RGB-LED: **Magenta/Lila** (Pins LEDR und LEDB auf LOW)
- **Seitheben** (lateralraise / LateralRaise / LateralRaises):
- RGB-LED: **Türkis** (Pins LEDB und LEDG auf LOW)
- **Saubere Ausführung Bizeps-Curl** (curl / curl_sauber):
- RGB-LED: **Grün** (Pin LEDG auf LOW)
- **Fehlerhafte Ausführung / Sonstiges (z. B. fehler_rotation, fehler_ellbogen):**
- RGB-LED: **Rot** (Pin LEDR auf LOW)

Requirements

FA2.4 – Das Gerät gibt durch die LED den Verbindungsstatus aus.

FA2.4.1 – Ansteuerung der RGB-LED zur Signalisierung von Fehlern (Rot), perfekten Curls (Grün), Seitheben (Türkis), Schulterdrücken (Lila) und Idle/Verbindungs Aufbau (Blau).

Kontrollfluss



4.1.5 BLE-Streamer

Diese Komponente überträgt die erfassten 6-Achsen-Sensordaten (Beschleunigung und Rotation) in Echtzeit per Bluetooth Low Energy (BLE) an die Mobile App.

C4-Architektur-Ebene

- **C4-Ebene:** Component
- **Deployable:** Nein (Läuft als Teil des Sensor Firmware Containers)

Beschreibung

Der BLE-Streamer initialisiert den nRF52840-Bluetooth-Stack, stellt einen GATT-Service mit einer IMU-Characteristic bereit und sendet die gefilterten Sensor-Messwerte (Beschleunigung in X, Y, Z und Drehraten in X, Y, Z) als binären 24-Byte-Puffer (6 float-Werte) an verbundene Clients.

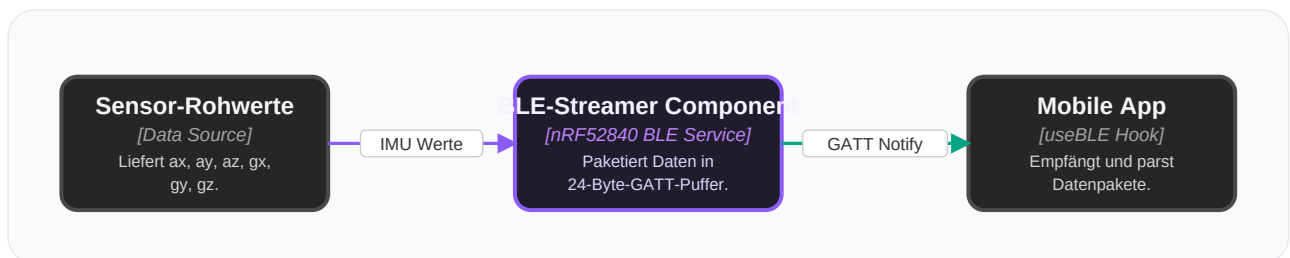
Requirements

FA2.5 – Das Gerät versendet die Daten an die App.

FA2.5.1 – Aufbau und Aufrechterhaltung einer stabilen Bluetooth LE Verbindung.

FA2.5.2 – Serialisierung und Übertragung der Sensordaten mit minimalem Overhead.

Datenfluss



- **GATT Profile & UUIDs***
- **Service-UUID:** 12345678-1234-1234-1234-123456789012
- **Characteristic-UUID:** 12345678-1234-1234-1234-123456789013 (BLERead | BLENotify)
- **Paketformat:** 24 Bytes (6 float-Werte, IEEE 754 float32, little-endian):
[accelX, accelY, accelZ, gyroX, gyroY, gyroZ]

4.1.6 Gehäuse (Enclosure)

Diese Komponente beschreibt das physische, schützende 3D-Druck-Gehäuse des Sensors.

C4-Architektur-Ebene

- **C4-Ebene:** Component
- **Deployable:** Nein (Physische Schutzhülle, keine Software-Ausführung)

Beschreibung

Das Gehäuse umschließt den XIAO-Mikrocontroller sowie die Peripherie (Display, Batterie). Es bietet Befestigungslaschen für ein standardmäßiges 20-mm-Sportarmband, um den Sensor stabil am Arm des Nutzers zu fixieren.

Requirements

FA2.6 – Das Gerät ist in einem Gehäuse.

FA2.6.1 – Schutz der Elektronik vor äußeren Einflüssen.

FA2.6.2 – Befestigungsmöglichkeit (z.B. Sportarmband) für stabile Fixierung während des Trainings.

Technische & Physische Parameter

- **Gesamtmaße:** 48 mm (Länge) x 24 mm (Breite) x 16 mm (Höhe)
- **Außenwandstärke:** 2.0 mm
- **Schließmechanismus:** Schnapp-Deckel (Lippe & Snap Bumps mit 0.2 mm Toleranz)
- **Komfort:** Abgerundete Ecken (Bevel-Breite 1.5 mm), um Druckstellen beim Tragen zu vermeiden.
- **Aussparungen:** Integrierter USB-C-Port zur Programmierung und Akku-Ladung.

Datenfluss



4.1.7 Abwägungen & Architekturentscheidungen

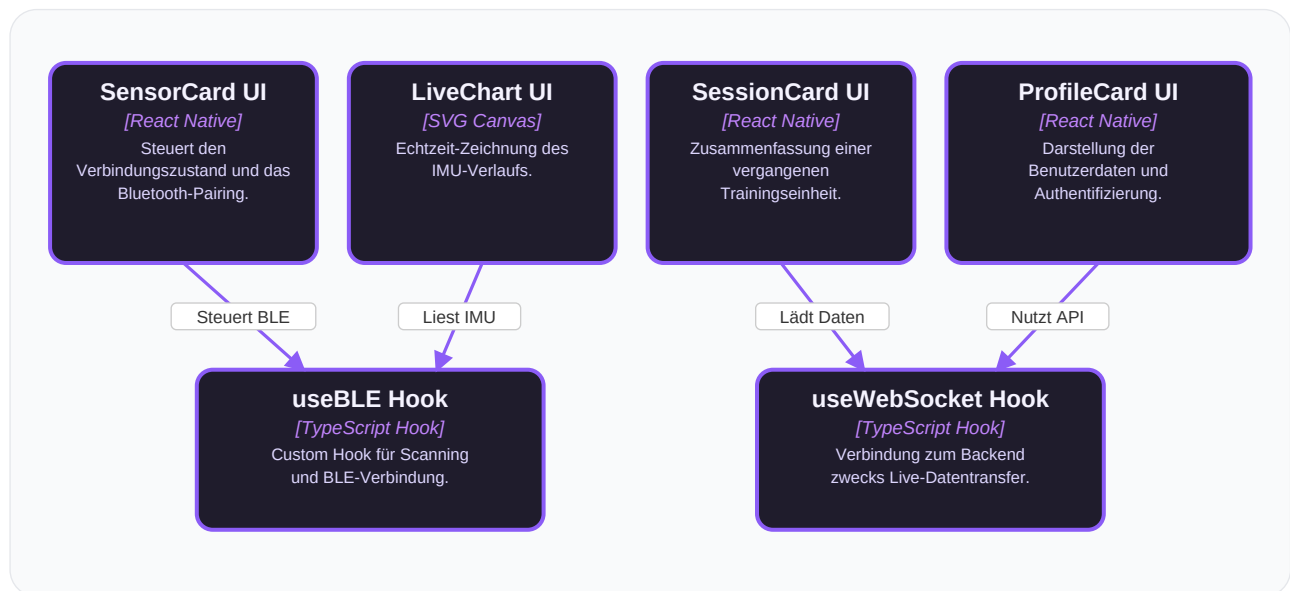
- **Lokale Auswertung vs. Cloud-Streaming:** Das Ausführen der Inferenz-Engine direkt auf dem Xiao-Controller minimiert die Latenz (**NF1**) und spart Bandbreite bei der Funkübertragung.
- **Energiebedarf:** Die kontinuierliche Sensordatenerfassung und BLE-Funkübertragung verbrauchen Energie, weshalb die Akkulaufzeit durch einen stromsparenden Betrieb im Idle und Deaktivieren nicht benötigter Hardware-Peripherie optimiert wird.

4.2 Mobile App (React Native)

Die Mobile App bildet das Benutzer-Interface (UI) des Systems. Sie verbindet sich per BLE mit der Xiao MCU, visualisiert Live-Bewegungsdaten und synchronisiert sie mit dem Backend.

Architektur-Abbildung (Komponenten)

Das folgende Komponenten-Diagramm veranschaulicht die Benutzeroberflächen-Module und Custom React Hooks des App-Containers:



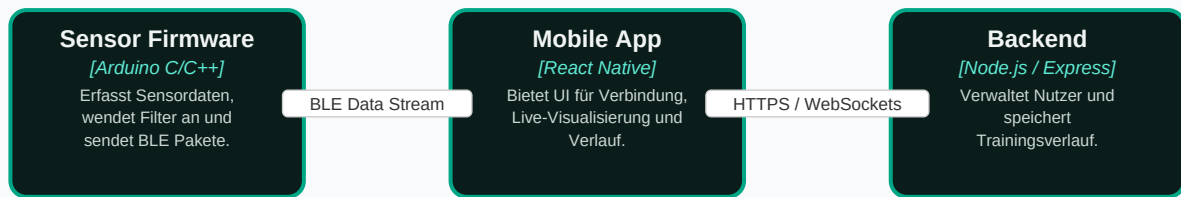
Dieses Dokument beschreibt die Mobile App als eigenständige, deploybare Einheit im C4-Modell.

C4-Architektur-Ebene

- **C4-Ebene:** Container
- **Deployable:** Ja
- **Deployment-Artefakt:** Android Package (.apk) / iOS IPA
- **Technologie-Stack:** React Native, Expo, TypeScript, Zustand, BLE PLX

Beschreibung

Die MoveLink Mobile App ist die primäre Benutzerschnittstelle des Systems. Sie läuft auf Android- und iOS-Endgeräten und verbindet sich über Bluetooth Low Energy (BLE) mit dem embedded Sensor-Gerät, um Bewegungsdaten in Echtzeit zu erfassen, zu visualisieren und zur persistenten Speicherung an das Backend zu übertragen.



Requirements

FA1.1 – Die App bietet eine Navigationsstruktur (Reiter/Menü) für Hardwaregeräte, Trainings und vergangene Einheiten.

FA1.1.1 – Die App bietet eine Seiten-Navigation (Drawer/SideNav) auf Tablets und Web.

FA1.1.2 – Die App bietet eine untere Tableiste (Tabs) auf Mobiltelefonen.

FA1.2 – Die App ermöglicht das Scannen nach verfügbaren Bluetooth-Sensorgeräten.

FA1.3 – Die App baut eine stabile Bluetooth-Verbindung zum Sensor auf.

FA1.4 – Die App demonstriert grafisch die auszuführende Übungsausführung (z. B. via Lottie-Animation).

FA1.5 – Die App visualisiert die empfangenen Sensordaten und den Übungsfortschritt in Echtzeit.

FA1.6 – Die App zeigt historische Trainingseinheiten grafisch und statistisch an.

FA1.7 – Die App zeigt Profildetails des Benutzers an.

Komponenten in diesem Container

Die App enthält mehrere Komponenten (C4-Komponenten-Ebene):

1. **[SideNav]**(file:///c:/Users/erlin/repo/movelink/app/components/SideNav.tsx): Navigationskomponente für die App-Steuerung. (Erfüllt: **FA1.1**)
2. **[SensorCard]**(file:///c:/Users/erlin/repo/movelink/app/components/SensorCard.tsx): Verwaltung der BLE-Geräteverbindung und Pairing. (Erfüllt: **FA1.2**, **FA1.3**, **NF3**)
3. **[LiveChart]**(file:///c:/Users/erlin/repo/movelink/app/components/LiveChart.tsx): Echtzeit-Visualisierung der IMU-Beschleunigungs- und Gyroskopwerte. (Erfüllt: **FA1.5**)

4. **[SessionCard](file:///c:/Users/erlin/repo/movelink/app/components/SessionCard.tsx)**: Visualisierung historischer Trainingseinheiten. (Erfüllt: **FA1.6**)
5. **[BLE-Hook (useBLE)](file:///c:/Users/erlin/repo/movelink/app/hooks/useBLE.ts)**: Kapselt die Bluetooth-Gerätekommunikation und den Reconnect. (Erfüllt: **FA1.3**, **NF2**)
6. **[ProfileCard](file:///c:/Users/erlin/repo/movelink/app/components/ProfileCard/architecture.md)**: Visualisierung der Benutzerprofildetails. (Erfüllt: **FA1.7**)

Architekturentscheidung (ADR / Abwägungen)

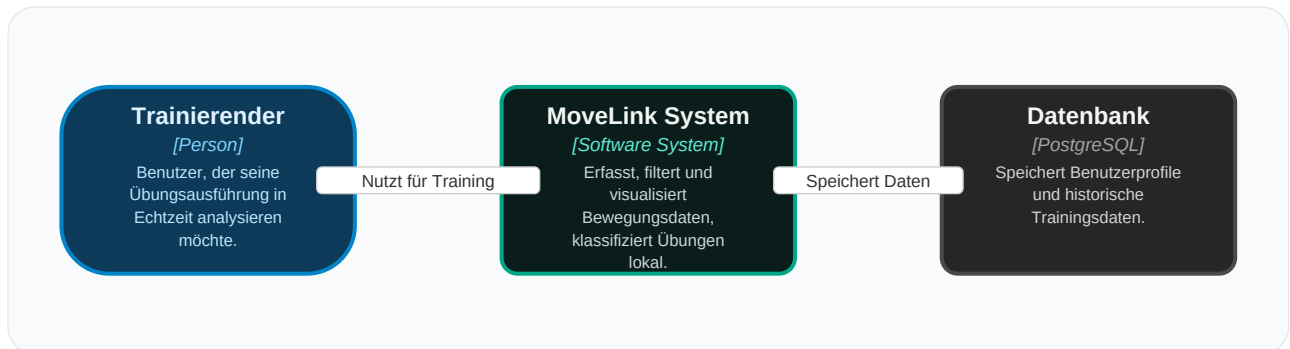
Entscheidung: Verwendung von **React Native (Hybrid-App)** zur plattformübergreifenden Entwicklung mit hoher Code-Wiederverwendbarkeit und schnellen UI-Iterationen, bei gleichzeitiger Optimierung des SVG-Diagramm-Renderings für den 50Hz Datenstrom.

4.3 Datenbank & Backend (PostgreSQL & Node.js)

Der Datenbank- und Backend-Container dient als zentraler Speicher und API-Gateway des Gesamtsystems. Er nimmt Profile und Trainingshistorien auf und stellt diese persistent bereit.

System-Kontext-Diagramm (C4 Level 1)

Das folgende System-Kontext-Diagramm zeigt die Position des MoveLink-Systems in seiner Betriebsumgebung, die Interaktion mit dem Trainierenden sowie die Verbindung zur persistenten Datenbank:



<!--

C4-Ebene: Container

Deployable: Ja

• ->

Dieses Dokument beschreibt die Datenbank und den Backend-Service als eigenständigen Container im C4-Modell.

C4-Architektur-Ebene

- **C4-Ebene:** Container
- **Deployable:** Ja
- **Deployment-Artefakt:** Docker Image / Cloud Service
- **Technologie-Stack:** Node.js, Express, PostgreSQL / SQLite

Beschreibung

Der Datenbank- und Backend-Container dient als zentraler Datenspeicher und API-Gateway für die MoveLink-Applikation. Er nimmt Trainingsdaten und Profile auf, validiert Benutzer und stellt sicher, dass historische Bewegungsdaten zur späteren Analyse persistent abgelegt werden.

Requirements

FA3.1 – Das System speichert und verifiziert Benutzerprofile und Anmeldeinformationen persistent.

FA3.2 – Das System speichert aufgezeichnete Trainingseinheiten (Übungstyp, Wiederholungen, Qualität) persistent zur historischen Auswertung.

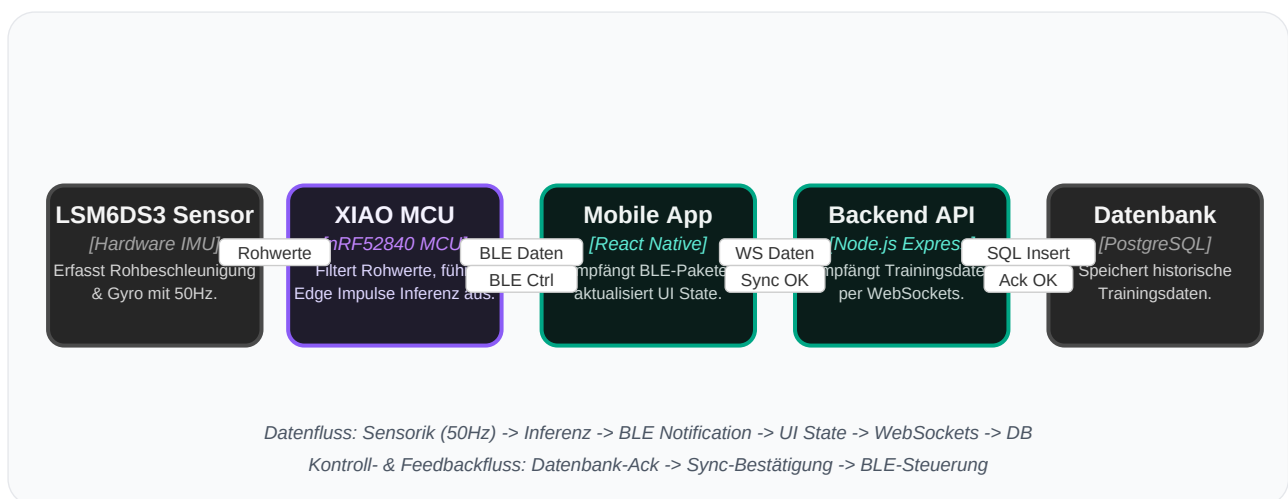
Komponenten in diesem Container

Die API- und Datenbankschnittstelle besteht aus folgenden logischen Komponenten:

1. **ProfileController / Auth Service:** Authentifiziert Benutzer und liefert Profildaten. (Erfüllt: **FA3.1**)
2. **Trainings-DB / Session Store:** Speichert und indiziert abgeschlossene Repetitions- und Sessiondaten. (Erfüllt: **FA3.2**)

System-Daten- & Kontrollfluss-Diagramm

Das folgende Diagramm visualisiert den End-to-End Daten- und Kontrollfluss im Gesamtsystem:



Architekturentscheidung (ADR / Abwägungen)

Entscheidung: Verwendung einer **PostgreSQL-Datenbank** zur persistenten und relationalen Ablage von Trainingsdaten und Nutzerprofilen. Das Backend wird mit **Node.js & Express** realisiert, um hohe Parallelität zu gewährleisten und asynchrone WebSocket-Verbindungen für Live-Streaming-Daten ressourcenschonend zu verarbeiten.

5. Rückverfolgbarkeits-Matrix (Traceability)

Die folgende Matrix veranschaulicht die Beziehungen und Verknüpfungen zwischen den Use Cases, den funktionalen Anforderungen (FA), den Systemkomponenten und den entsprechenden Klassen/Code-Dateien im System. Klicken Sie auf eine Anforderungs-ID, um zu deren Definition zu springen, oder auf eine Komponente, um deren C4-Modell-Kapitel aufzurufen.

| Use Case | Map | FA Ebene 1 | Sub-FA | Component | Detail-FA Ebene 3 | Klassenebene |
|----------|-----|------------|--------|-----------|-------------------|--------------|
|----------|-----|------------|--------|-----------|-------------------|--------------|

| | | | | | | |
|---|--|--|--------------|---|---|--|
| UC-1
Trainingsgerät verbinden |  | FA1
Applikation (Mobile App) | FA1.1 | SideNav | FA1.1.1 Die App bietet eine Seiten-Navigation (Drawer/SideNav) auf Tablets und Web. | (Zielstruktur) |
| | | | | | FA1.1.2 Die App bietet eine untere Tableiste (Tabs) auf Mobiltelefonen. | (Zielstruktur) |
| | | | FA1.2 | SensorCard | - | Props </> StatusLabel()
</> ScanningDots()
</> SensorCard() </> |
| | | | FA1.3 | SensorCard | - | isConnected </> StatusLabel()
isActive </> ScanningDots() </>
style SensorCard() </> |
| | | | FA1.4 | - | - | isConnected </>
isActive </> color </>
styles |
| | | | FA1.5 | LiveChart | - | Props </> HEIGHT </>
buildPaths() </> allVals
</> min </> max </> |
| | | | FA1.6 | SessionCard | - | Props </> xStep </>
formatDuration() </> first
</> last </> |
| UC-2
Echtzeit-Training überwachen | | FA2
Trainingsgerät (Sensor-Firmware) | FA1.7 | ProfileCard | - | isConnected </>
isActive </>
style SensorCard() </>
styles |
| | | | FA2.1 | Sensordatenerfassung (Loop) | FA2.1.1 Beschleunigung wird eingelesen

FA2.1.2 Drehung wird eingelesen | styles chartW
series
paths </> hasData
</> minVal </> maxVal

</> switchMode() </>
styles (Zielstruktur) |
| | | | FA2.2 | Inferenz-Engine (Edge Impulse) | FA2.2.1 Das Gerät erkennt einen Idle Modus | (Zielstruktur) |
| | | | | | FA2.2.2 Das Gerät erkennt einen Curl | (Zielstruktur) |
| | | | | | FA2.2.3 Das Gerät erkennt einen shoulder press | (Zielstruktur) |
| | | | | | FA2.2.4 Das Gerät erkennt einen Lateral raise | (Zielstruktur) |
| | | | | | FA2.2.5 Das Gerät erkennt eine tricep extension | (Zielstruktur) |
| | | | FA2.3 | Inferenz-Engine (Edge Impulse) | - | (Zielstruktur) |
| | | | FA2.4 | LED- & Display-Controller | FA2.4.1 Ansteuerung der RGB-LED zur Signalisierung von Fehlern (Rot), perfekten | (Zielstruktur) |
| | | | FA2.5 | BLE-Streamer | FA2.5.1 Ansteuerung der RGB-LED zur Signalisierung von Fehlern (Rot), perfekten | (Zielstruktur) |
| | | | | | FA2.5.2 Ansteuerung der RGB-LED zur Signalisierung von Fehlern (Rot), perfekten | (Zielstruktur) |
| | | | | | FA2.5.3 Ansteuerung der RGB-LED zur Signalisierung von Fehlern (Rot), perfekten | (Zielstruktur) |
| UC-3
Trainingsdaten einsehen | | FA3
Datenbank & Backend | FA2.6 | Gehäuse | FA2.6.1 Schutz der Elektronik vor äußeren Einflüssen.

FA2.6.2 Befestigungsmöglichkeit (z.B. Sportarmband) für stabile Fixierung während des Trainings. | (Zielstruktur) |
| | | | FA3.1 | ProfileController / Auth Service | - | (Zielstruktur) |
| | | | FA3.2 | Trainings-DB / Session Store | - | (Zielstruktur) |